

Algorithm Visualization - Don't Teach, Let'em Play

Luděk Kučera
Charles University
Prague, Czech Republic

Abstract

The poster describes how experience with the author's algorithm visualization environment (AVE), Algovision (www.algovision.org), was used to propose and implement a four-layer AVE architecture. The most innovative part of the architecture — the lowest layer, called the animator layer — allows learners to perform atomic visual actions “manually” using a mouse and other interface devices. The main intention was to fully implement the principles of Cognitive Constructivism, exemplified in algorithm visualization (AV) by the (Extended) Engagement Taxonomy, which the author considers one of the most appropriate epistemological approaches to AV.

The benefits of the animator layer in the new architecture are demonstrated through an experiment involving two groups of high school students, who were learning algorithms for binary search trees — a topic typically taught in undergraduate data structures and algorithms (DSA) courses — and working independently with the AVE, guided only indirectly by the instructor.

ACM Reference Format:

Luděk Kučera, Charles University, Prague, Czech Republic. 2025. Algorithm Visualization - Don't Teach, Let'em Play. In *Proceedings of the ACM Global Computing Education Conference 2025 Vol 2 (CompEd 2025)*, October 21–25, 2025, Gaborone, Botswana. ACM, New York, NY, USA, 2 pages. <https://doi.org/10.1145/3736251.3747319>

The first algorithm visualizations (AV) emerged as early as the 1980s, and the decade from 1995 to 2005 is often considered the golden age of AV. During this period, there were high hopes that AV would revolutionize the teaching and learning of algorithms. However, studies evaluating the effectiveness of AV — many of which appeared around the same time — produced rather pessimistic results.

A typical outcome of early AV assessments can be illustrated by the following citation from Stasko, Badre, and Lewis [8]: “We embarked on this study seeking to gather concrete empirical evidence that algorithm animations assist learning. The evidence did not appear.”

Later studies — for example, Hundhausen, Douglas, and Stasko [3] — identified learner engagement as the principal factor influencing the effectiveness of AV. Passive viewing of algorithm animations was found to have little to no educational benefit; the more actively the learner was involved, the more effective the learning process became. This line of research culminated in the formulation of the Engagement Taxonomy (ET) by Naps et al. [6], see also Šimonák [9]:

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for third-party components of this work must be honored. For all other uses, contact the owner/author(s).

CompEd 2025, Gaborone, Botswana

© 2025 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-1942-4/2025/10

<https://doi.org/10.1145/3736251.3747319>

- (1) **No viewing:** No visualization, only textual material.
- (2) **Viewing:** Visualization with no or limited interaction.
- (3) **Responding:** Visualization with content-related questions.
- (4) **Changing:** Changing or modifying the visualization.
- (5) **Constructing:** Constructing students' own visualization.
- (6) **Presenting:** Visualizations presented to others.

The author views the Engagement Taxonomy (ET)—later extended by Myller et al. [5] into the Extended Engagement Taxonomy (EET)—as an effort to rediscover the principles of Cognitive Constructivism (CC), encapsulated in the idea that “learners do better by doing than by viewing” (as paraphrased in the title of this submission). The author's experience with Algovision (www.algovision.org) supports the view that CC is the most suitable epistemological approach to algorithm visualization (AV).

Unfortunately, most algorithm visualizations—including those presented in [1, 2, 7]—remain at the lower levels of the Engagement Taxonomy (ET). One notable exception was TRAKLA [4] (now discontinued), which was used primarily for assessing student knowledge. TRAKLA supported ‘algorithm simulation’, allowing learners to manually execute the animation step by step.

Moreover, even the complete ET/EET frameworks do not fully realize the Cognitive Constructivist (CC) emphasis on *learning by doing*. A learning process guided by ET/EET in the context of algorithm visualization has two key aspects: the transfer of knowledge (in this case, knowledge of an algorithm) and the learner's engagement in the process. While engagement is significantly enhanced at the higher levels of ET/EET, the knowledge transfer remains tied to the temporal behavior of the algorithm. In this model, the learner's role in the transfer process is passive, limited to acquiring knowledge presented by a (human or automated) instructor.

Motivated by prior experience with Algovision, the author has developed a four-layer AVE architecture:

- **Animator layer:** Presents atomic visual actions—such as moving a token from one node to another, inserting or deleting a node, rotating a tree edge, or changing/flipping a node's color—that can be triggered either by the upper layers of the AVE or *directly by the learner*, typically through mouse interaction.
- **Algorithm layer:** Generates visualizations of specific algorithms (e.g., insertions or deletions in BSTs or red-black trees, shortest path algorithms) by scripting calls to the animator layer.
- **Scene layer:** Prepares scenes that illustrate specific cases or properties of the algorithms defined in the previous layer. This layer also manages control elements, input generation, and editing.
- **Guide layer:** Provides high-level automated visualization of an entire lecture by sequencing scenes, invoking algorithm animations, and concurrently displaying explanatory text and synthesized voice narration.

While the algorithm, scene, and guide layers were already implicitly present in Algovision, the principal contribution of the

layered AVE architecture is the introduction of a distinct animator layer, which affords learners full interactivity. It enables them to execute algorithms manually, investigate behavioral variants, and explore “what if” questions and their consequences (e.g., rotating a red-black tree edge with a black child node).

Although we are still discovering how to fully leverage the potential of the animator layer, three advantageous modes of use have been identified.

The first, which the author refers to as *playgrounds*, was already partially present in Algovision and served as the main inspiration for developing the layered architecture. Playgrounds are designed for lab or homework assignments and aim to build skills in working with primitive operations. For example, a binary search tree (BST) edge rotation playground presents a binary tree where clicking on an edge rotates it. Students solve exercises such as promoting a given node to the root or balancing an unbalanced tree.

The second use case, perhaps available in TRAKLA as well, is scaffolded algorithm simulation, where a learner first watches an automatic step-by-step animation of an algorithm, then gradually begins performing selected steps manually, and ultimately becomes capable of executing the entire algorithm without system assistance.

However, the author has found that the most effective way to use the new AVE architecture is to enable students to discover algorithms independently, guided by instructors who assign carefully designed tasks aligned with a structured learning trajectory.

This approach was validated in an experiment involving two groups of high school students learning binary search tree algorithms—typically covered in undergraduate data structures and algorithms (DSA) courses—who worked independently with the AVE while receiving only indirect guidance from the instructor.

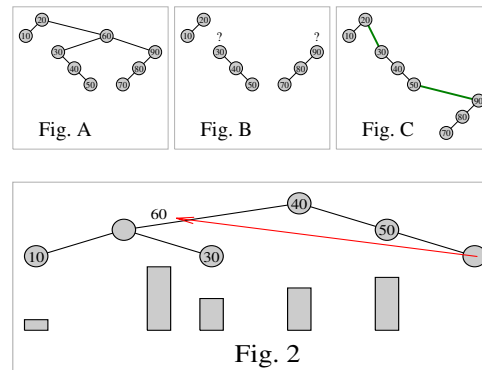
The students were asked, in a round-robin order, to solve simple problems assigned by the instructor, with their work displayed on a large screen visible to the entire lecture room. A carefully designed sequence of exercises led the groups to discover the BST search, insert, and delete algorithms within 30 minutes for one group and 32 minutes for the other. Out of several dozen problems, only one student’s answer was incorrect—and it was immediately corrected by other students.

As an example that clearly illustrates the benefits of this method, we present the initial steps involved in deleting a key from a BST node with two children.

First, a student was asked to *remove* a node with two children, which resulted in two subtrees separated from the main tree (see Fig. 1A,B). The next student was tasked with reconnecting the trees while maintaining the correct key order. In both groups, after reconnecting the root of one subtree to the parent of the removed node, the student—much to our surprise—was able to reconnect the second subtree as well (see Fig. 1C).

This unplanned development demonstrated that students can be very creative when provided with sufficiently powerful tools. In our case, the instructor then had to explain that this method can significantly increase the tree height; students who already understood BST search and insertion were able to correctly explain why maintaining a low tree height is desirable.

After completing this “what if” branch, the next student was asked to remove the key and explore possible replacement keys by trying all keys in the tree. The animator layer of the AVE enables



moving keys using the mouse. To help students, our visualization represents node keys graphically as columns below the nodes (see Fig. 2). In both groups, students had no difficulty identifying that only two keys could be moved to the keyless node without violating the BST key order.

The simpler remaining steps of the process are not described.

Conclusion

Our results clearly show that to actively engage learners in the cognitive transfer of algorithm knowledge, an AVE must provide tools for active interaction with the entire visual landscape surrounding the algorithm—not just the algorithm itself. To the author’s best knowledge, no prior AVE offers this level of functionality.

Algovision already employs 378 different animators, so transforming its visualization system into the layered architecture of Algovision-2.0 is technically straightforward. This primarily involves adding a direct user interface to the animators and updating how they are combined into visualizations. Binary search trees visualization is already functional; AVL trees, B-trees, and red-black trees have been tested; and standard, binomial, and Fibonacci heaps are coming soon. Sorting, graph algorithms, and more advanced algorithms such as FFT, the Simplex Algorithm, and the Conjugate Gradient Method are expected to be added within the next year.

References

- [1] David Galles. Data structure visualization. <https://www.cs.usfca.edu/~galles/visualization>, 2006.
- [2] Steven Halim. Visualgo. <https://visualgo.net>, 2011.
- [3] Christopher D. Hundhausen, Sarah A. Douglas, and John T Stasko. A meta-study of algorithm visualization effectiveness. *Journal of Visual Languages and Computing*, 13:259–290, 2002.
- [4] L. Malmi, V. Karavirta, A. Korhonen, J. Nikander, O. Seppälä, and P. Silvasti. Visual algorithm simulation exercise system with automatic assessment: Trakla2. *Informatics Educ*, 3(2):267–288, 2004.
- [5] Niko Myller, Roman Bednarik, Erkki Sutinen, and Mordechai Ben-Ari. Extending the engagement taxonomy: Software visualization and collaborative learning. *ACM Transactions on Computing Education (TOCE)*, 9(1):1–27, 2009.
- [6] T. Naps, G. Röfling, V. Almström, W. Dann, R. Fleischer, C. Hundhausen, A. Korhonen, L. Malmi, M. McNally, S. Rodger, and J. Velázquez-Iturbide. Exploring the role of visualization and engagement in computer science education. *SIGCSE Bull*, 35(2):131–152, 2003.
- [7] Clifford A. Shaffer. Opendsa: An interactive etextbook for computer science courses. In *SIGCSE '16*, page 5, 2016.
- [8] John T. Stasko, Albert Badre, and Clayton Lewis. Do algorithm animations assist learning? an empirical study and analysis. In *ACM INTERCHI'93 Conference on Human Factors in Computing Systems*, pages 61–66. ACM Press, New York, 1993.
- [9] Slavomír Šimóniák. Increasing the engagement level in algorithms and data structures course by driving algorithm visualizations. *Informatica*, 44(3):327–334, 2020.